

Question 1: Decision Trees and Dimensionality Reduction

Part A

Step 1: Load the dataset and split into training and testing sets

```
In [4]: from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target
feature_names = data.feature_names
target_names = data.target_names

#70-30 train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
print("Class names:", target_names)
```

```
Training set shape: (398, 30)
Testing set shape: (171, 30)
Class names: ['malignant' 'benign']
```

Step 2: Train a Decision Tree Classifier

```
In [8]: from sklearn.tree import DecisionTreeClassifier

# Train decision tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
```

```
Out [8]: ▼ DecisionTreeClassifier ⓘ ⓘ
          ► Parameters
```

Step 3: Evaluate the model using Accuracy, F1, Precision, and Recall

```
In [9]: from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score

# Predict on test set
y_pred = dt_model.predict(X_test)

# Compute evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print("Part A Results")
print("Accuracy :", accuracy)
print("F1 Score :", f1)
print("Precision:", precision)
print("Recall   :", recall)
```

Part A Results

Accuracy : 0.9181286549707602

F1 Score : 0.9345794392523364

Precision: 0.9345794392523364

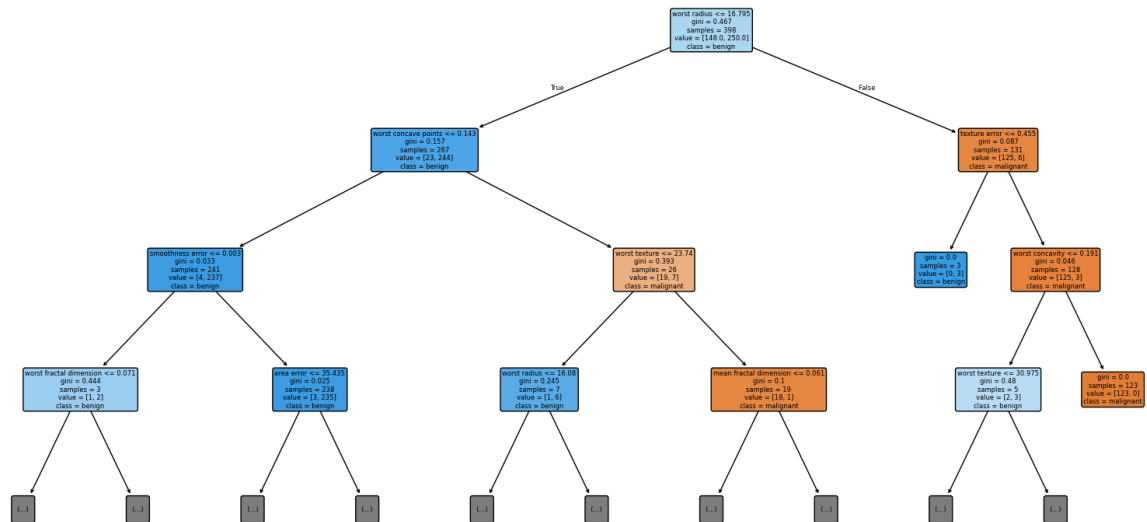
Recall : 0.9345794392523364

Step 4: Visualize the trained Decision Tree

```
In [10]: import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 10))
plot_tree(
    dt_model,
    feature_names=feature_names,
    class_names=target_names,
    filled=True,
    rounded=True,
    max_depth=3 # limit depth for cleaner visualization
)
plt.title("Decision Tree (Top 3 Levels)")
plt.show()
```

Decision Tree (Top 3 Levels)



Root node explanation

```
In [11]: root_index = dt_model.tree_.feature[0]
root_feature = feature_names[root_index]
root_threshold = dt_model.tree_.threshold[0]

print("Root feature:", root_feature)
print("Root threshold:", root_threshold)
```

Root feature: worst radius

Root threshold: 16.795000076293945

Explanation

The first split is on worst radius.

This means the decision tree found that this feature provides the best first separation between malignant and benign tumors. In decision trees, the root node is chosen because it gives the largest reduction in impurity, usually measured by:

Gini impurity, or Entropy (depending on settings)

Here, worst radius was selected because it most effectively separates the two classes early in the tree.

Part B

Step 1: Standardize the features

```
In [15]: from sklearn.preprocessing import StandardScaler
```

```

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

Step 2: Apply PCA and retain 90% variance

```

In [16]: from sklearn.decomposition import PCA

pca = PCA(n_components=0.90, random_state=42)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Original number of features:", X_train.shape[1])
print("Reduced number of features:", X_train_pca.shape[1])
print("Explained variance retained:", pca.explained_variance_ratio_.sum())

```

Original number of features: 30
Reduced number of features: 7
Explained variance retained: 0.9189135045926773

Step 3: Train a new Decision Tree on PCA-transformed data

```

In [18]: dt_pca_model = DecisionTreeClassifier(random_state=42)
dt_pca_model.fit(X_train_pca, y_train)

```

```

Out[18]:
DecisionTreeClassifier
Parameters

```

Step 4: Evaluate the PCA-based Decision Tree

```

In [19]: # Predict using PCA-based model
y_pred_pca = dt_pca_model.predict(X_test_pca)

# Compute metrics
accuracy_pca = accuracy_score(y_test, y_pred_pca)
f1_pca = f1_score(y_test, y_pred_pca)
precision_pca = precision_score(y_test, y_pred_pca)
recall_pca = recall_score(y_test, y_pred_pca)

print("Part B Results")
print("Accuracy :", accuracy_pca)
print("F1 Score :", f1_pca)
print("Precision:", precision_pca)
print("Recall   :", recall_pca)

```

Part B Results

Accuracy : 0.9122807017543859

F1 Score : 0.9261083743842364

Precision: 0.9791666666666666

Recall : 0.8785046728971962

Step 4: Part A vs Part B Comparison

```
In [20]: print("Comparison of Results")
print("-" * 40)
print(f"{'Metric':<12}{'Part A':<15}{'Part B':<15}")
print("-" * 40)
print(f"{'Accuracy':<12}{accuracy:<15.4f}{accuracy_pca:<15.4f}")
print(f"{'F1 Score':<12}{f1:<15.4f}{f1_pca:<15.4f}")
print(f"{'Precision':<12}{precision:<15.4f}{precision_pca:<15.4f}")
print(f"{'Recall':<12}{recall:<15.4f}{recall_pca:<15.4f}")
```

Comparison of Results

Metric	Part A	Part B
Accuracy	0.9181	0.9123
F1 Score	0.9346	0.9261
Precision	0.9346	0.9792
Recall	0.9346	0.8785

Findings and interpretation

1. Yes, there was a small performance drop overall after PCA.
2. PCA reduced the feature space from 30 dimensions to 7 dimensions. This can help by: (i) reducing computational complexity, (ii) simplifying the feature space (iii) potentially reducing noise and redundancy. However, for a decision tree, dimensionality reduction is not always beneficial because trees naturally perform feature selection through splitting. So in this case, training may become slightly simpler but predictive quality did not improve.
3. Interpretability became worse after PCA. Before PCA, the tree splits on real medical features. For example, the root node was worst radius. This is easy to explain to a human. After PCA however, the tree splits on Principal Component 1, Principal Component 2, etc. Each principal component is a mathematical combination of many original variables. This makes it much harder to explain what the split means in real-world terms. So PCA helps reduce dimensionality, but it sacrifices the straightforward interpretability of the decision tree.

Question 2: A Comparative Study of Clustering Algorithms

Part A: Finding the right "K"

Import libraries, generate and visualize the datasets

```
In [35]: import numpy as np

from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans
```

```
In [36]: # Dataset 1: Blobs
X_blobs, y_blobs = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=0.60,
    random_state=0
)

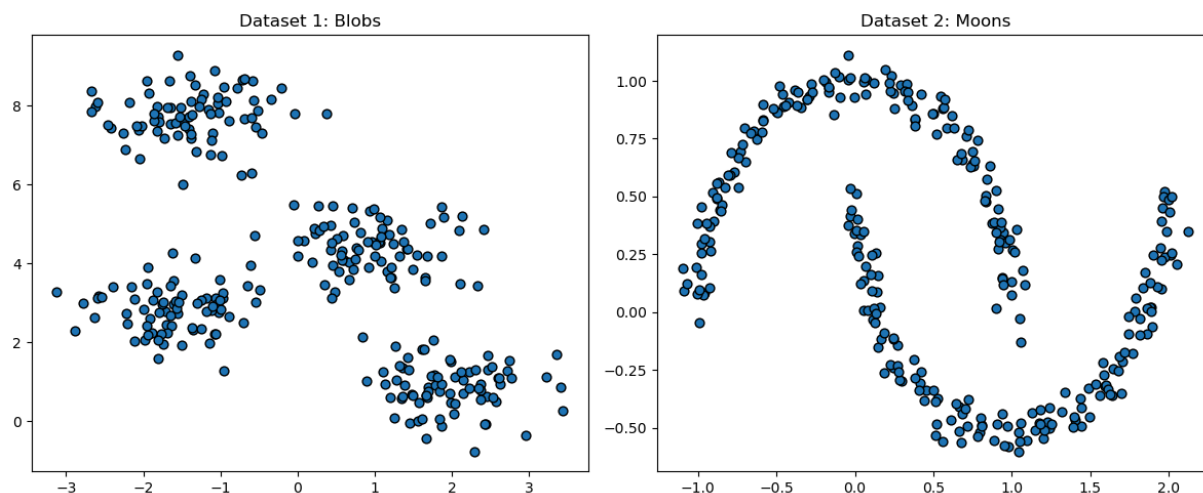
# Dataset 2: Moons
X_moons, y_moons = make_moons(
    n_samples=300,
    noise=0.05,
    random_state=0
)
```

```
In [37]: plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X_blobs[:, 0], X_blobs[:, 1], s=40, edgecolor='k')
plt.title("Dataset 1: Blobs")

plt.subplot(1, 2, 2)
plt.scatter(X_moons[:, 0], X_moons[:, 1], s=40, edgecolor='k')
plt.title("Dataset 2: Moons")

plt.tight_layout()
plt.show()
```

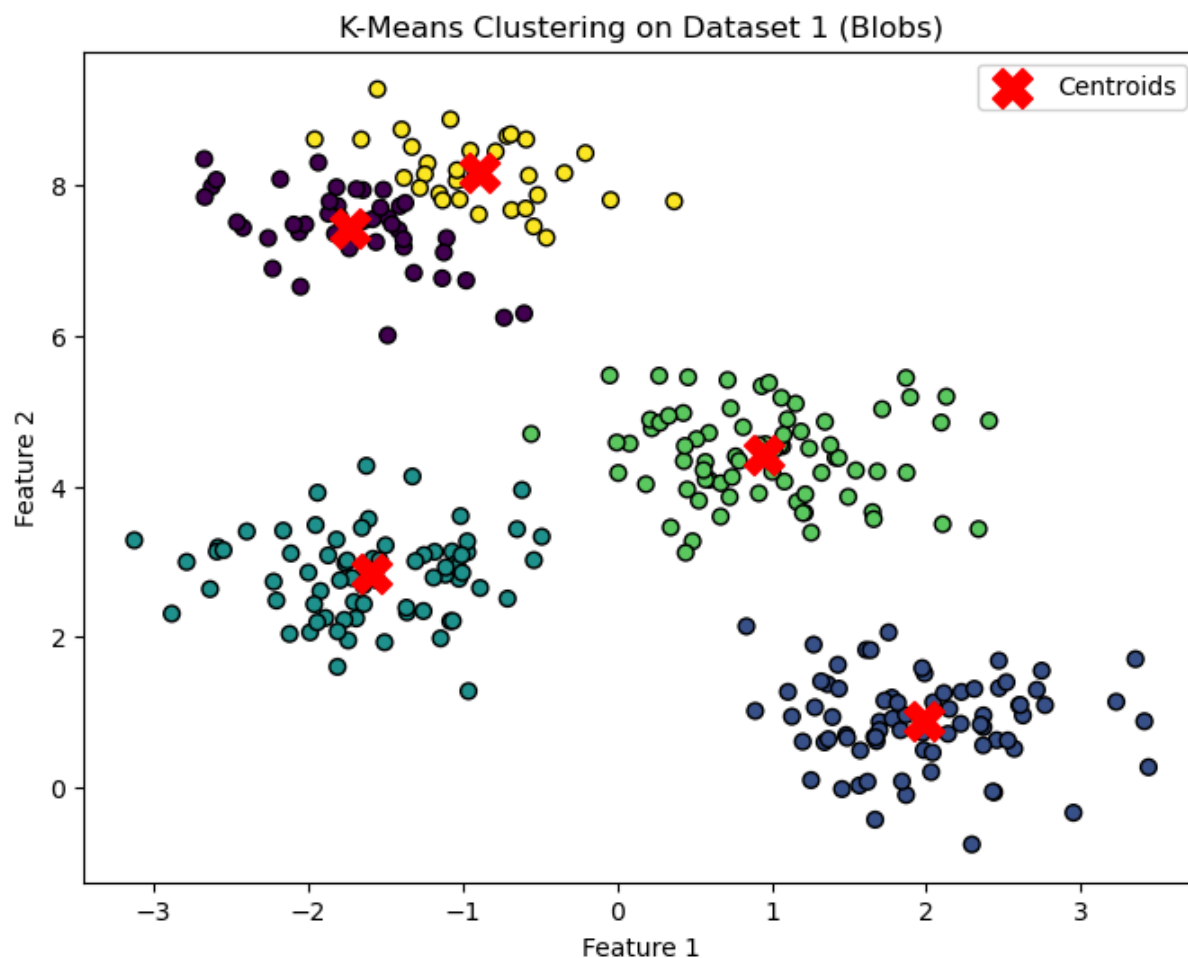


1. Apply K-Means Clustering to Dataset 1 (Blobs) (without elbow method)

```
In [38]: # Apply K-Means
kmeans_blobs = KMeans(n_clusters=5, random_state=0, n_init=10)
blob_labels = kmeans_blobs.fit_predict(X_blobs)

# Centroids
centroids = kmeans_blobs.cluster_centers_

# Plot the clustered data
plt.figure(figsize=(8, 6))
plt.scatter(X_blobs[:, 0], X_blobs[:, 1], c=blob_labels, s=40, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=250, marker='X', label='Centroids')
plt.title("K-Means Clustering on Dataset 1 (Blobs)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

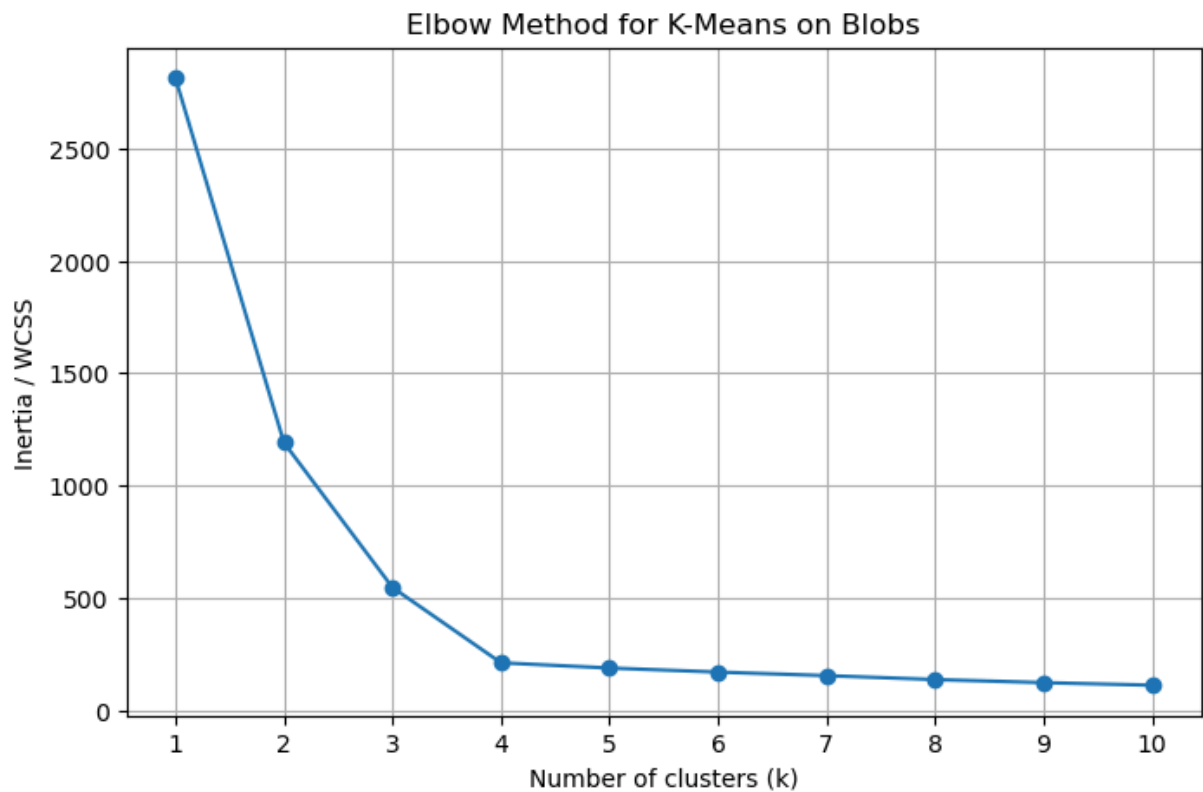


2. Use the Elbow Method on the Blobs dataset

```
In [25]: inertias = []
k_values = range(1, 11)

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)
    kmeans.fit(X_blobs)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(k_values, inertias, marker='o')
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia / WCSS")
plt.title("Elbow Method for K-Means on Blobs")
plt.xticks(k_values)
plt.grid(True)
plt.show()
```

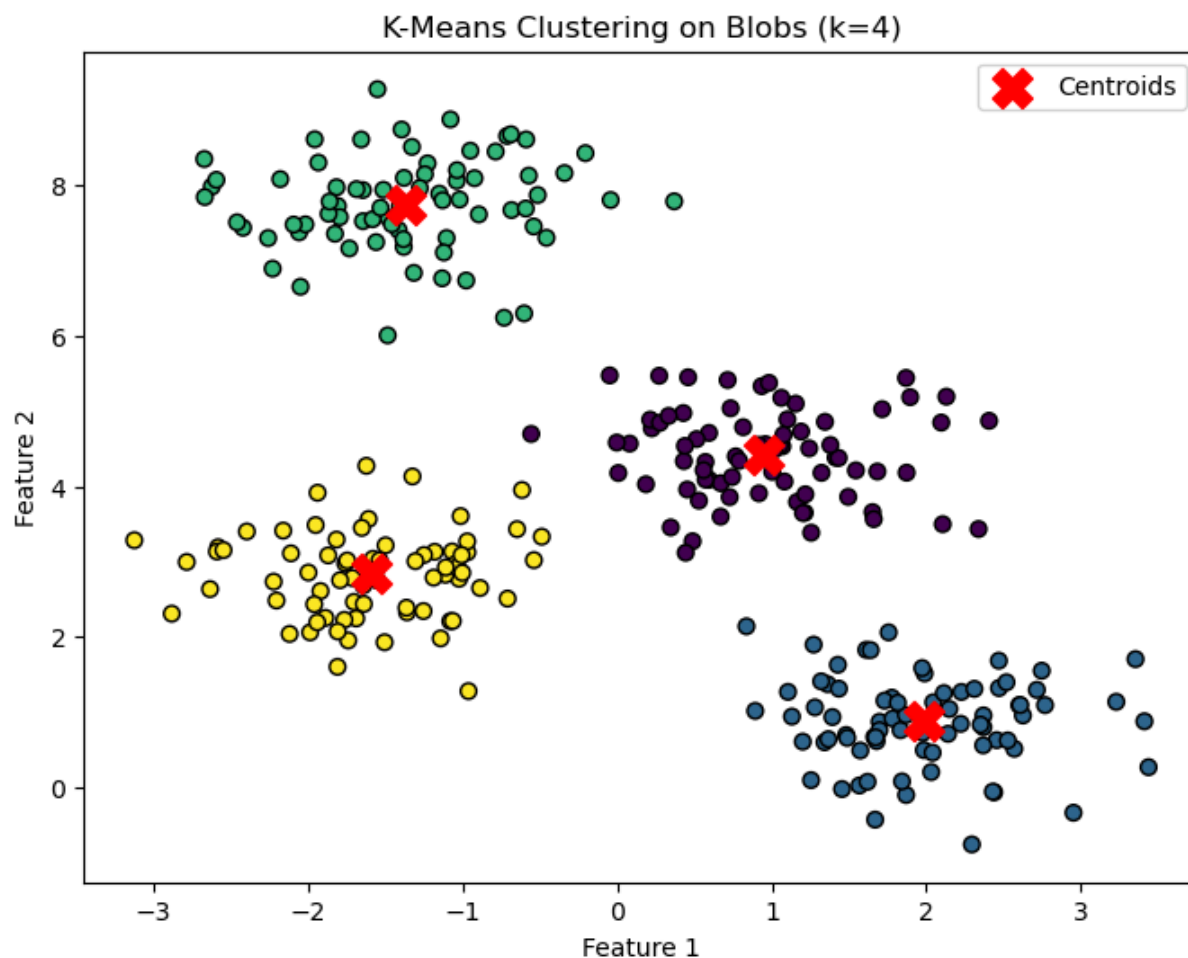



3. Train K-Means using the chosen optimal K = 4

```
In [26]: optimal_k = 4

kmeans_blobs = KMeans(n_clusters=optimal_k, random_state=0, n_init=10)
blob_labels = kmeans_blobs.fit_predict(X_blobs)
centroids = kmeans_blobs.cluster_centers_

plt.figure(figsize=(8, 6))
plt.scatter(X_blobs[:, 0], X_blobs[:, 1], c=blob_labels, s=40, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=250, marker='X', label='Centroids')
plt.title("K-Means Clustering on Blobs (k=4)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

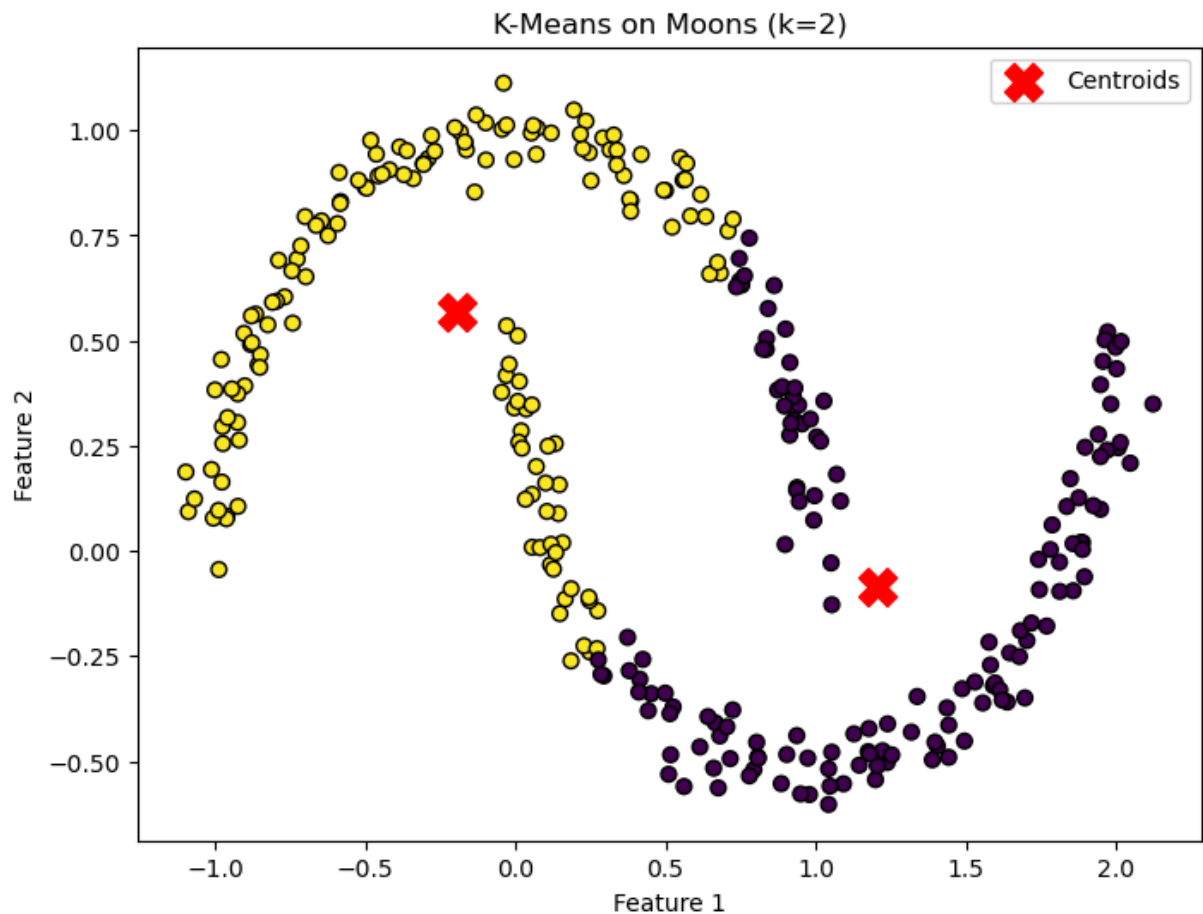


Part B: When Centroids Fail

Step 1: Apply K-Means to the Moons dataset

```
In [27]: kmeans_moons = KMeans(n_clusters=2, random_state=0, n_init=10)
moon_kmeans_labels = kmeans_moons.fit_predict(X_moons)

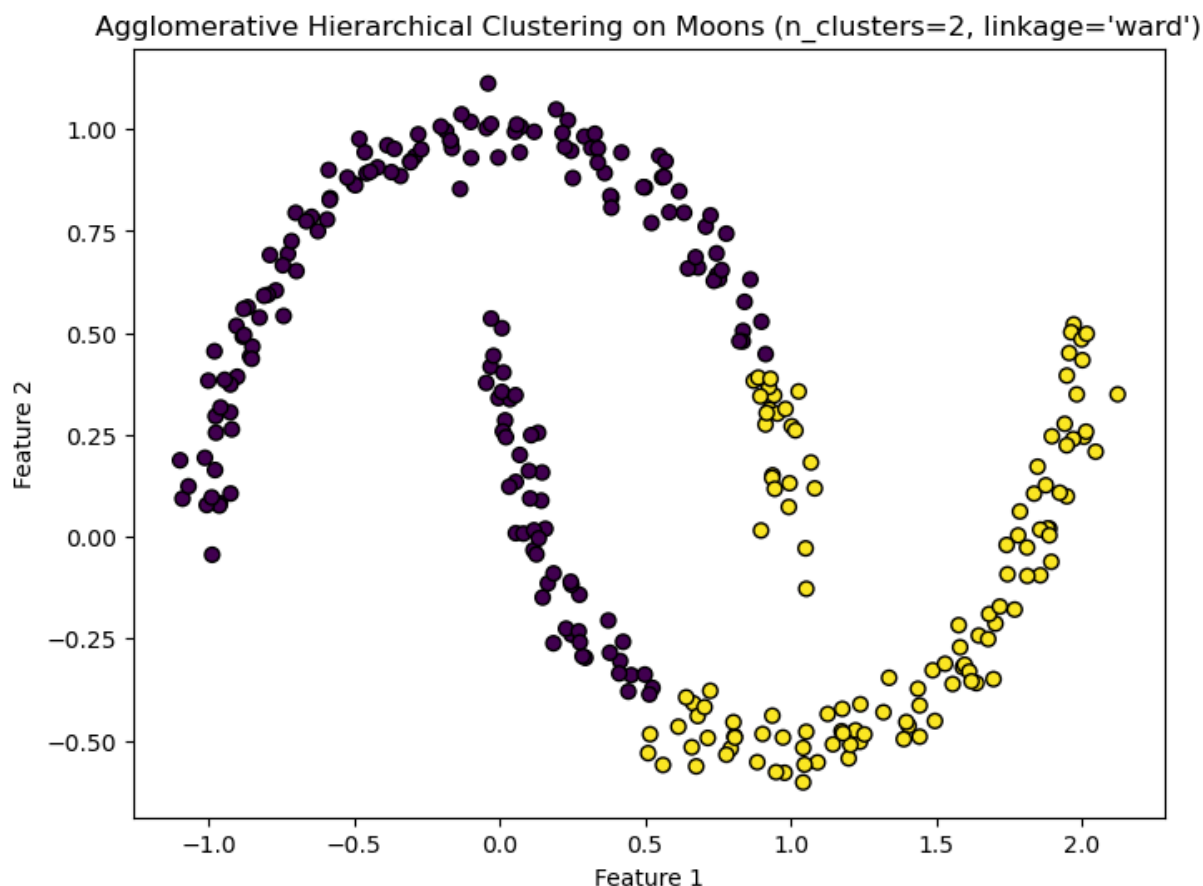
plt.figure(figsize=(8, 6))
plt.scatter(X_moons[:, 0], X_moons[:, 1], c=moon_kmeans_labels, s=40, cmap='
plt.scatter(kmeans_moons.cluster_centers[:, 0], kmeans_moons.cluster_center
            c='red', s=250, marker='X', label='Centroids')
plt.title("K-Means on Moons (k=2)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```



Step 2: Apply Agglomerative Hierarchical Clustering to the Moons dataset

```
In [28]: aggro = AgglomerativeClustering(n_clusters=2, linkage='ward')
moon_agglo_labels = aggro.fit_predict(X_moons)

plt.figure(figsize=(8, 6))
plt.scatter(X_moons[:, 0], X_moons[:, 1], c=moon_agglo_labels, s=40, cmap='v')
plt.title("Agglomerative Hierarchical Clustering on Moons (n_clusters=2, linkage='ward')")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Explanation of findings:

K-Means did not successfully separate the two moon-shaped clusters. Instead, it divided the dataset into two regions based on centroid distance, which is not suitable for curved shapes. Agglomerative Hierarchical Clustering with Ward linkage also struggled because Ward linkage favors compact clusters and minimizes within-cluster variance. Since the moon shapes are non-convex and curved, both algorithms fail to capture the true geometry of the data.

Part C: DBSCAN to the Rescue

Experiment with DBSCAN parameters

```
In [30]: dbscan = DBSCAN(eps=0.2, min_samples=5)
moon_dbscan_labels = dbscan.fit_predict(X_moons)

print("Unique cluster labels:", np.unique(moon_dbscan_labels))
```

Unique cluster labels: [0 1]

Plot final DBSCAN result

```

In [31]: unique_labels = np.unique(moon_dbscan_labels)

plt.figure(figsize=(8, 6))

for label in unique_labels:
    if label == -1:
        plt.scatter(
            X_moons[moon_dbscan_labels == label, 0],
            X_moons[moon_dbscan_labels == label, 1],
            s=40,
            c='black',
            marker='x',
            label='Noise'
        )
    else:
        plt.scatter(
            X_moons[moon_dbscan_labels == label, 0],
            X_moons[moon_dbscan_labels == label, 1],
            s=40,
            label=f'Cluster {label}'
        )

plt.title("DBSCAN on Moons")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()

```

